

Como levantar el proyecto con Docker en cualquier computador

Guia practica para Windows, macOS y Linux. No requiere instalar Java, Node.js, Maven ni MySQL.

Componente	Contenedor	Funcion
Frontend	uifce-frontend	React compilado y servido por Nginx.
Backend	uifce-backend	API Spring Boot sobre Java 21.
Base de datos	uifce-mysql	MySQL 8 con volumen persistente.

Resultado: la aplicacion queda disponible en <http://localhost:8080> (o el puerto configurado en FRONTEND_PORT).

Tiempo esperado del primer arranque: 3 a 10 minutos, porque Docker descarga imagenes y Maven/npm descargan dependencias dentro de las etapas de build.

1. Requisitos previos

Cada persona necesita solamente Git y Docker con Docker Compose v2.

Sistema	Instalar	Comprobacion
Windows 10/11	Docker Desktop + Git. Activar WSL2 cuando Docker lo solicite.	docker version
macOS	Docker Desktop + Git.	docker version
Linux	Docker Engine + plugin docker-compose + Git.	docker version

Comprobar desde una terminal:

```
git --version
docker version
docker compose version
```

- Docker Desktop debe estar abierto en Windows/macOS.
- En Linux, el servicio Docker debe estar activo y el usuario debe tener permiso para usarlo.
- Se recomiendan al menos 4 GB de RAM libres v 5 GB de espacio.

No instalar: Java, Maven, Node, npm o MySQL. Los Dockerfiles ya contienen esas herramientas.

2. Obtener el proyecto

```
git clone https://github.com/jacordobah/ingesoftII.git
cd ingesoftII
git switch devops
git pull origin devops
```

Todas las instrucciones siguientes deben ejecutarse desde la carpeta raíz, donde esta docker-compose.yml.

3. Crear el archivo .env

Docker Compose lee variables desde un archivo .env ubicado en la raíz del repositorio. Este archivo no se sube a Git.

Crear .env con este contenido:

```
SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/uifce_support
SPRING_DATASOURCE_USERNAME=root
SPRING_DATASOURCE_PASSWORD=cambiar-esta-clave
SPRING_PROFILES_ACTIVE=production
MYSQL_ROOT_PASSWORD=cambiar-esta-clave
MYSQL_DATABASE=uifce_support
MYSQL_PORT=3306
FRONTEND_PORT=8080
GOOGLE_CLIENT_ID=local-disabled
GOOGLE_CLIENT_SECRET=local-disabled
```

Si van a probar Google OAuth: sustituyan los dos valores local-disabled por credenciales validas y registren <http://localhost:8080/login/oauth2/code/google> como URI de redireccion autorizada en Google Cloud.

Sin credenciales Google reales, el backend puede iniciar y el login tradicional puede utilizarse; el boton Google no funcionara.

Creacion rapida en Windows

- Se puede ejecutar scripts\start.bat, que crea/completa .env y levanta Docker.
- Despues, agregar manualmente GOOGLE_CLIENT_ID y GOOGLE_CLIENT_SECRET si la plantilla no los contiene.

Creacion rapida en macOS/Linux

```
cp scripts/.env.example .env
# Editar .env y agregar GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET
```

4. Construir y levantar los contenedores

Comando normal para todos los sistemas:

```
docker compose up -d --build
```

Que sucede:

- Se descarga MySQL 8.
- El backend se compila con JDK 21 y se ejecuta sobre una imagen JRE ligera.
- El frontend se compila con Node 22 y se copia a Nginx.
- Se crea el volumen mysql_data, por lo que los datos sobreviven al reinicio.

Ver el estado:

```
docker compose ps
```

Estado	Significado	Accion
Up / healthy	Servicio listo.	Continuar.
Up / starting	Healthcheck todavia esperando.	Esperar 30-90 segundos.
unhealthy	El proceso corre, pero fallo su healthcheck.	Revisar logs y contingencia.
Exited	El proceso termino con error.	docker compose logs

5. Limitacion conocida de esta version

El healthcheck del backend consulta un endpoint que ahora requiere autenticacion. Por ello puede marcar unhealthy aunque Spring Boot haya iniciado correctamente, y Compose puede no arrancar el frontend.

Contingencia segura para esta version:

```
docker compose up -d --build mysql backend
docker compose logs backend
# Cuando aparezca 'Started ApiApplication':
docker compose up -d --no-deps frontend
```

Luego abrir <http://localhost:8080>. El estado unhealthy puede permanecer hasta que el proyecto cambie el healthcheck por un endpoint publico dedicado.

6. Verificar que todo funciona

```
docker compose ps
docker compose logs --tail=100 backend
docker compose logs --tail=100 frontend
docker compose logs --tail=100 mysql
```

Prueba	Resultado esperado
Abrir http://localhost:8080	Se muestra la pagina de inicio de sesion.
docker compose logs backend	Mensaje 'Started ApiApplication' sin excepcion final.
docker compose logs mysql	ready for connections.
Recargar navegador	Nginx devuelve la SPA y las rutas se mantienen.

7. Operaciones diarias

Objetivo	Comando
Iniciar sin reconstruir	docker compose up -d
Ver logs en vivo	docker compose logs -f
Ver logs del backend	docker compose logs -f backend
Reiniciar backend	docker compose restart backend
Aplicar cambios de codigo	docker compose up -d --build
Detener contenedores	docker compose down
Actualizar desde Git	git pull origin devops; docker compose up -d --build

Los datos no se borran con docker compose down. Permanecen en el volumen mysql_data.

Borrado total de la base de datos

Solo cuando se quiera comenzar desde cero:

```
docker compose down -v
```

Este comando elimina los contenedores y el volumen MySQL. Los datos no se pueden recuperar salvo que exista backup.

8. Solucion de problemas

Problema	Diagnostico	Solucion corta
Puerto 8080 ocupado	Error bind/port allocated.	Cambiar FRONTEND_PORT=8081.
Puerto 3306 ocupado	MySQL local ya usa el puerto.	Cambiar MYSQL_PORT=3307.
Backend no inicia	docker compose logs backend	Revisar .env, Google vars y conexion MySQL.
Frontend no inicia	Backend unhealthy.	Usar --no-deps frontend tras confirmar Started ApiApplication.
Cambie .env y no aplica	Contenedor conserva entorno anterior.	docker compose up -d --force-recreate.
Build usa cache viejo	Cambios no aparecen.	docker compose build --no-cache; docker compose up -d.
Permiso Docker en Linux	permission denied docker.sock	Configurar grupo docker o usar sudo temporalmente.
Login Google falla	redirect_uri_mismatch	Registrar URI exacta con puerto correcto.
Datos inconsistentes	Migracion o volumen antiguo.	Respaldar; luego down -v solo si se acepta borrar.

Comandos de diagnostico completos

```
docker compose config
docker compose ps -a
docker compose logs --tail=200
docker inspect uifce-backend
docker system df
```

9. Lista de comprobacion para compartir

- Docker y Git instalados.
- Rama devops actualizada.
- .env creado y fuera de Git.
- Contraseñas locales cambiadas.
- Variables Google presentes, reales o local-disabled.
- docker compose up ejecutado desde la raiz.
- Backend muestra Started ApiApplication.
- Aplicacion abre en el puerto configurado.
- Nunca compartir .env, secretos OAuth ni dumps de base de datos

Comando de rescate: docker compose logs --tail=200. Adjuntar esa salida al pedir ayuda, eliminando antes cualquier secreto.